Analog-to-Digital Converter (ADC)

Prof. Gerardo Flores

April 2026

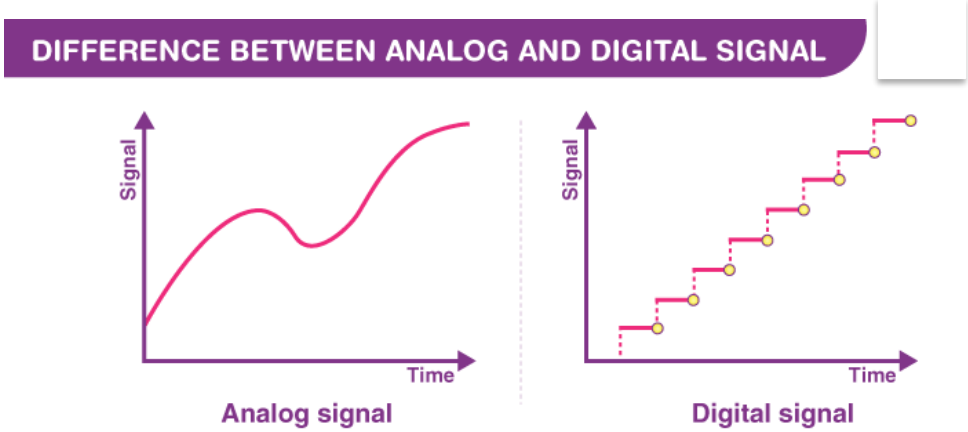
Microprocessor Systems (CSCE 3340 - SENG 3345)

TAMIU

What is an ADC?

Analog-to-Digital Converter (ADC)

- Converts an **analog voltage signal** (e.g. from a sensor or potentiometer) into a **digital number**.
- Essential when microcontrollers need to read **non-digital signals**.
- Output is a binary value proportional to the input voltage.



Key ADC Parameters

Resolution:

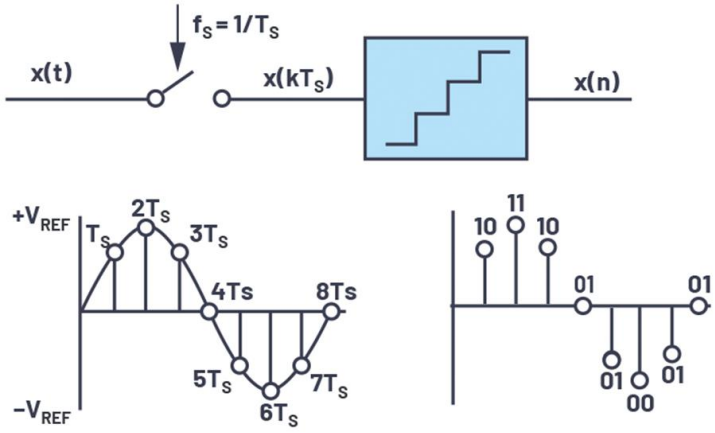
- Determines the number of steps in the conversion.
- STM32 supports 12-bit (0–4095) [$2^{12} = 4096$], 10-bit (0–1023), etc.

Reference voltage (Vref):

- Defines the upper voltage limit.
- Typically 3.3 V in STM32 Nucleo boards.

Sampling time:

- Defines how long the ADC samples the signal before conversion.



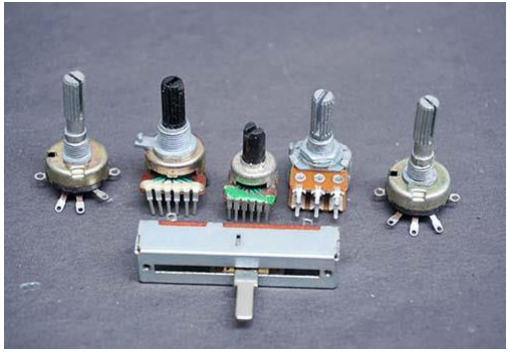
How STM32 ADC Works

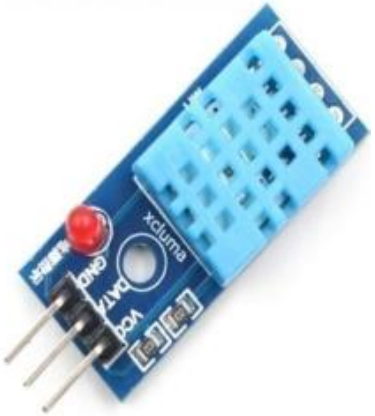
ADC is part of the **peripheral modules** of the STM32 microcontroller. Supports:

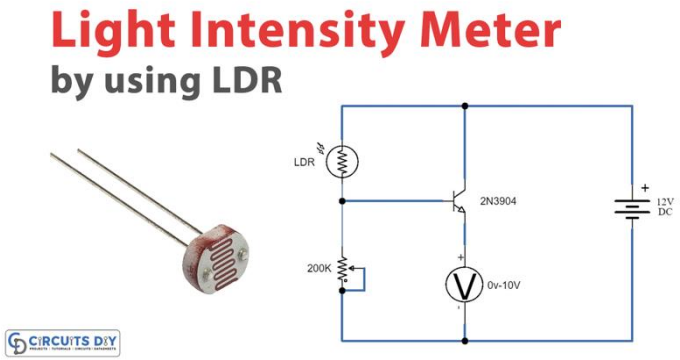
- **Single conversion** (one channel)
- **Scan mode** (multiple channels)
- **Interrupts or** Direct Memory Access (**DMA)** for efficient sampling

Typical Applications

- Reading **potentiometers**
- Measuring **temperature sensors (e.g. LM35, NTC)**
- Detecting **light levels** (via LDR)
- Reading **battery voltages**



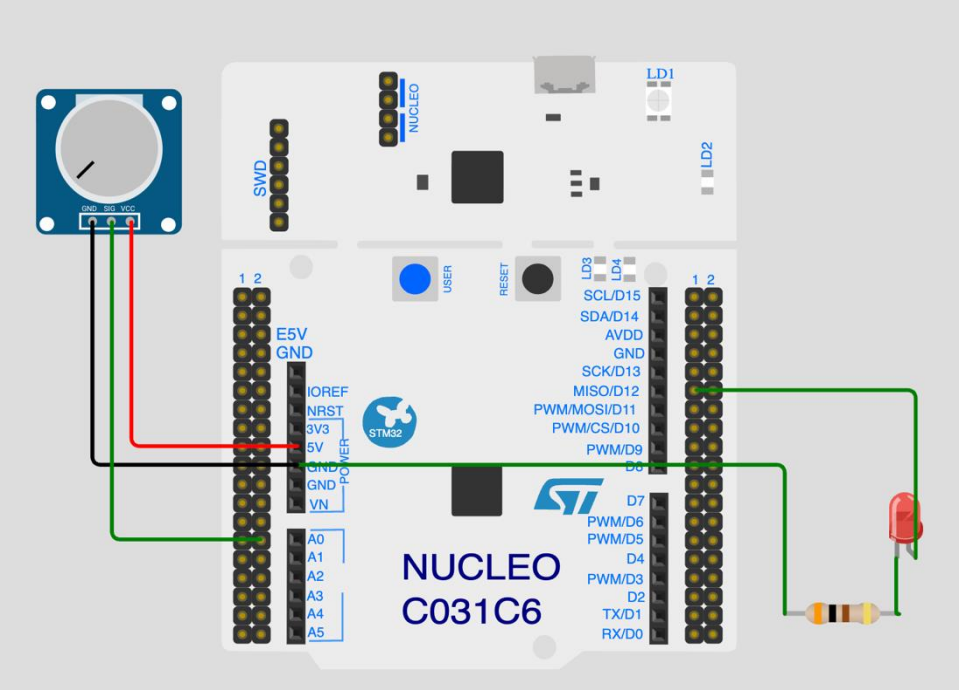




Minimal Recipe to Use an ADC (Register-Level STM32 Workflow)

ADCplusLED

1. Enable GPIO clock
2. Configure pin as analog
3. Enable ADC clock
4. Exit deep power-down
5. Enable ADC regulator
6. Wait stabilization delay
7. Run calibration
8. Select channel
9. Enable ADC
10. Wait until ready
11. Start conversion
12. Wait end-of-conversion
13. Read result



“Complex” steps

4. **Exit deep power-down**: Clearing the DEEPPWD bit exits deep-power-down mode and powers the ADC internal circuitry.

5. **Enable ADC regulator**: The ADC internal analog regulator provides a stable supply for accurate conversions.

7. **Run calibration**: Calibration compensates internal offset and mismatch errors [*offset error, capacitor mismatch, comparator bias*] in the ADC analog front-end.

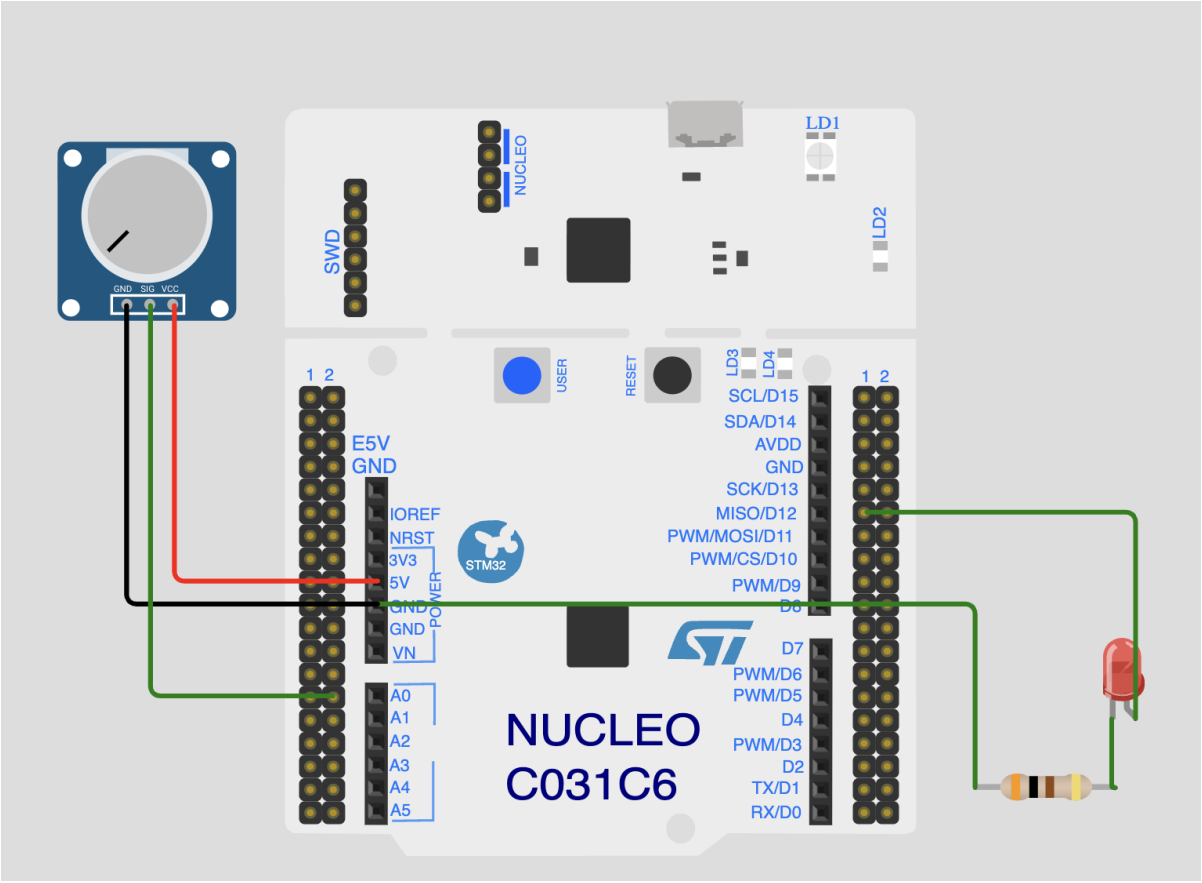
8. **Select channel**: The ADC can read multiple analog inputs. The CHSELR register selects which one is connected to the converter.

Example 1: Controlling LED Blink Rate Using ADC Input

This program demonstrates how to use the ADC (Analog-to-Digital Converter) of an STM32 microcontroller to control the blink rate of an LED based on an analog input, such as a potentiometer.

Description:

- The ADC reads the analog value from channel 0 [PA0].
- The read value (0 to 4095 for 12-bit resolution) is mapped to a delay range between **100 ms** and **2000 ms [it is an approximation]**.
- The LED connected to **GPIOB PA5** toggles its state according to this dynamic delay, effectively changing its blink speed based on the analog input.


GND SIG VCC
SWD
NUCLEO
LD1
LD2
USER
RESET
LD3
LD4
1 2
E5V
GND
IOREF
NRST
3V3
5V
GND
GND
VN
POWER
A0
A1
A2
A3
A4
A5
STM32
NUCLEO
C031C6
SCL/D15
SDA/D14
AVDD
GND
SCK/D13
MISO/D12
PWM/MOSI/D11
PWM/CS/D10
PWM/D9
D8
D7
PWM/D6
PWM/D5
D4
PWM/D3
D2
TX/D1
RX/D0

Initialization

```c
#include "stm32c031xx.h"

/* ===== ADC bits ===== */
#define ADC_CR_ADEN_BIT        (1U << 0)
#define ADC_CR_ADSTART_BIT     (1U << 2)
#define ADC_CR_ADVREGEN_BIT    (1U << 28)
#define ADC_CR_DEEPPWD_BIT     (1U << 29)
#define ADC_CR_ADCAL_BIT       (1U << 31)

#define ADC_ISR_ADRDY_BIT      (1U << 0)
#define ADC_ISR_EOC_BIT        (1U << 2)
```

Conversion

```c
/* ==========================================
   ADC single conversion
   ==========================================

static uint16_t adc_read(void)
{
    /* 10. Start one conversion by software */
    ADC1->CR |= ADC_CR_ADSTART_BIT;

    /* 11. Wait until conversion is complete */
    while (!(ADC1->ISR & ADC_ISR_EOC_BIT)) {
    }

    /* 12. Return 12-bit result: 0 to 4095 */
    return (uint16_t)ADC1->DR;
}
```

Configuration

```c
/* ==========================================
   ADC initialization: PA0 = analog input, channel 0
   ==========================================

static void adc_init(void)
{
    /* 1. Enable GPIOA clock */
    RCC->IOPENR |= RCC_IOPENR_GPIOAEN;

    /* 2. Set PA0 to analog mode */
    GPIOA->MODER |= (3U << 0);

    /* 3. Enable ADC clock */
    RCC->APBENR2 |= RCC_APBENR2_ADCEN;

    /* 4. Enable internal ADC voltage regulator */
    ADC1->CR |= ADC_CR_ADVREGEN_BIT;

    /* 5. Wait for regulator stabilization */
    small_delay(20000);

    /* 6. Start ADC calibration */
    ADC1->CR |= ADC_CR_ADCAL_BIT;

    /* Wait until calibration is complete */
    while (ADC1->CR & ADC_CR_ADCAL_BIT) {
    }

    /* 7. Select channel 0 (PA0) */
    ADC1->CHSELR = (1U << 0);

    /* 8. Enable ADC */
    ADC1->CR |= ADC_CR_ADEN_BIT;

    /* 9. Wait until ADC is ready */
    while (!(ADC1->ISR & ADC_ISR_ADRDY_BIT)) {
    }
}
```

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ADC EN	Res.	TIM17 EN	TIM16 EN	TIM15 EN
											rw		rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TIM14 EN	USART1 EN	Res.	SPI1 EN	TIM1 EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SYS CFG EN
rw	rw		rw	rw											rw

Bits 31:21 Reserved, must be kept at reset value.

Bit 20 **ADCEN**: ADC clock enable
Set and cleared by software.
0: Disable
1: Enable

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ADCAL	Res.	Res.	ADVREGEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
rs			rw												

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ADSTP	Res.	ADSTART	ADDIS	ADEN
											rs		rs	rs	rs

read / set by software
write 1 → it activates

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	CCRDY	Res.	EOCAL	Res.	AWD3	AWD2	AWD1	Res.	Res.	OVR	EOS	EOC	EOSMP	ADRDY
		rc_w1		rc_w1		rc_w1	rc_w1	rc_w1			rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

read / clear by writing 1

Project Idea:
Temperature Sensor Reading with Dual 7-Segment Displays

Design and implement a system using the STM32 microcontroller to **read a temperature sensor via ADC** and **display the temperature value** on **two 7-segment displays**.

- Properly **scale the ADC values** to match the sensor's temperature range.
- Ensure the system displays accurate readings, showing both **minimum (e.g., 00)** and **maximum values (e.g., corresponding to 99)** based on the sensor’s range.
- Use efficient GPIO management to control both displays.
- Bonus: Implement a simple refresh/multiplexing if needed for better display performance.

This project will help you practice ADC configuration, data processing, and interfacing with 7-segment displays.

Partial solution idea

ADC-Temperatura_baremetal

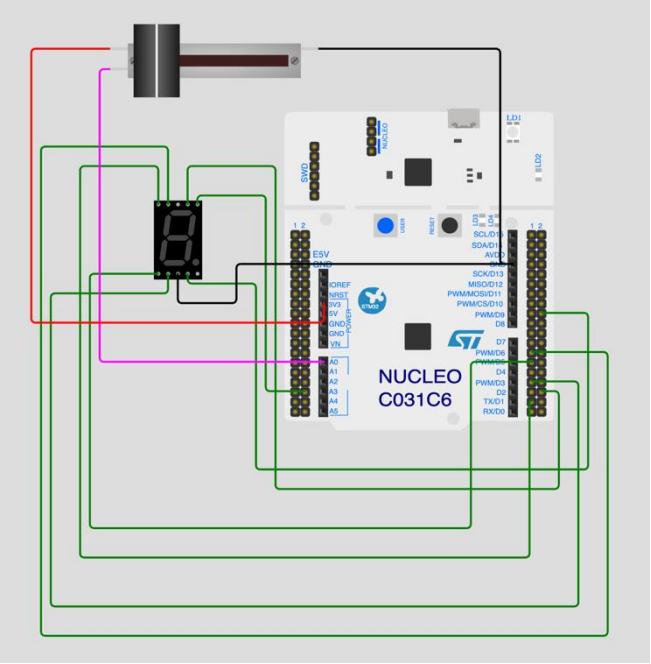
```c
int main(void)
{
    gpio_init();
    adc_init();

    while (1) {
        uint16_t adc_value = adc_read();

        /* Scale 0..4095 to 0..9 --> 10 possible values */
        uint8_t display_num = (adc_value * 10U) / 4095U;
        //uint8_t display_num = (adc_value * 10U) >> 12;
        // Map ADC range 0-4095 to display range 0-9 using a right shift (division by 4096)

        display_number(display_num);

        small_delay(120000);
    }
}
```



$$y = \left\lfloor \frac{x \cdot N}{2^{12}} \right\rfloor$$

2^12 = 4096

display_num = (adc_value * N) >> 12;

display_num = (adc_value * N) / 4096;

Summary

- ADCs let microcontrollers sense the **analog world**.
- STM32 provides flexible ADCs with HAL abstraction.
- Always match resolution and reference voltage to your application needs.
- Combine with PWM, sensors, and control systems for interactive applications.